Computational Physics

Summer 2026

Lecture #18

17. 6. 2026

Dylan Nelson

Numerical Errors

When solving ODEs, two main sources:

(1) truncation error

(2) round-off error

Def: the truncation error $\epsilon_t \sim |\tilde{y}_i - y_i|$ is the difference between the true and approximate solution

Ex: Euler's method: $\tilde{y}_{i+1} = \tilde{y}_i + h f(t_i, \tilde{y}_i) + \underbrace{\mathcal{O}(h^2)}_{\text{truncation error}}$

is the error from assuming that the curve $y(t)$ is a straight line between t_i, t_{i+1}

- exists even if computers did floating-point arithmetic at infinite accuracy...

Def: the round-off error is η the smallest possible number such that $1 + \eta \neq 1$ given the finite precision, discrete representation of numbers on the computer

$\Rightarrow$ every FP operation incurs $\epsilon_{ro} \sim \mathcal{O}(\eta)$

Ex: Euler's method, $N = \frac{b-a}{h} \sim \mathcal{O}(h^{-1})$ steps

- assume errors are cumulative (could be much worse)

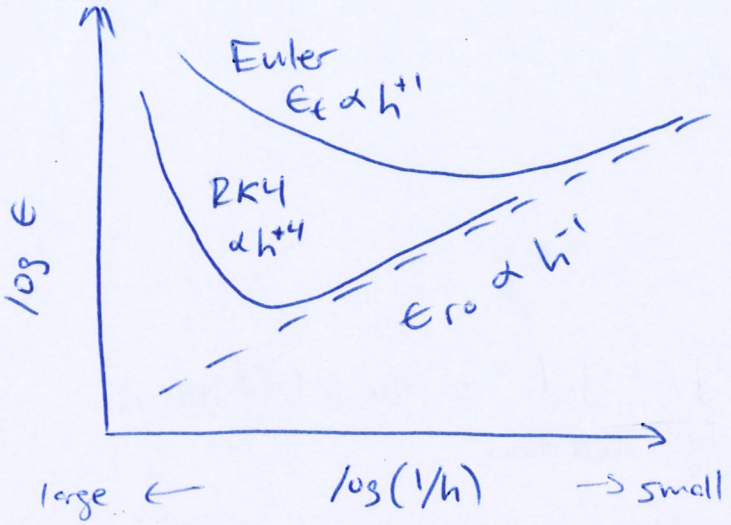
$\Rightarrow \epsilon_{ro} \sim \mathcal{O}(\eta/h)$

↑ as $h \to$ smaller, $\epsilon_{ro} \to$ bigger

Def: the total error is $\epsilon = \epsilon_t + \epsilon_{ro} \sim h + \eta/h$

ϵ_{ro} dominates for small h

ϵ_t dominates for large h



$\min(\epsilon) \sim \eta^{1/2}$

$\Rightarrow \boxed{h_{min} \sim \eta^{1/2}}$

two important implications of h_{min}:

(1) no point in taking $h < h_{min}$

$\Rightarrow$ increases work but not accuracy

(2) value of η is important for total ϵ

$\eta_{float} = 1.2 \times 10^{-7}$

$\Rightarrow h_{min} \sim 3 \times 10^{-4}$

$\sim \epsilon_{min}$

32 bit / 4 byte

"single" precision

$\eta_{double} = 2.2 \times 10^{-16}$

$\Rightarrow h_{min} \sim 10^{-8}$

$\sim \epsilon_{min}$

64 bit / 8 byte

"double" precision

{note: h_{min} values apply to Euler's method only!}

- ϵ_{min} of double usually OK, but ϵ_{min} of single usually not

also have:

- quad precision (128 bit)
- "arbitrary precision"

} implemented in software not hardware $\Rightarrow$ slow

Numerical (in) stability

Def: solution is unstable if $\lim_{i\to\infty} |\tilde{y}_i - y_i|$ diverges

i.e. numerical solution diverges from true solution

- all simple integration schemes are unstable if stepsize h is made sufficiently large

Ex: $y' = -\alpha y$ with $y(0) = 1$ and $\alpha > 0$

analytic solution:

$y(t) = \exp(-\alpha t)$ is monotonically decreasing with t

numerical solution:

- consider Euler's method, $t_i = ih$ for $i = 0, 1, 2, \ldots$

$$\Rightarrow \tilde{y}_{i+1} = \tilde{y}_i + h(-\alpha \tilde{y}_i)$$
$$= (1 - \alpha h)\,\tilde{y}_i$$
$$= (1 - \alpha h)^i\, y_0$$

prefactor determines if $\tilde{y}_{i+1}/\tilde{y}_i > 1$ or < 1
i.e. if our solution increases or decreases

(1) monotonically declines if:

$$0 < 1 - \alpha h < 1 \quad \Rightarrow \quad h < 1/\alpha$$

is the timestep constraint for stability

(2) oscillates, but overall declines:

$$-1 < 1 - \alpha h < 0 \quad \Rightarrow \quad 1/\alpha < h < 2/\alpha$$

(3) increases!

$$1 - \alpha h < -1 \quad \Rightarrow \quad h > 2/\alpha$$

result is unstable
i.e. total garbage

Ex: consider instead the backward Euler method:

$$\tilde{y}_{i+1} = \tilde{y}_i + h y'_{i+1}$$

$$= \tilde{y}_i + h f(t_{i+1}, \tilde{y}_{i+1})$$

$$= \tilde{y}_i - \alpha h \tilde{y}_{i+1}$$

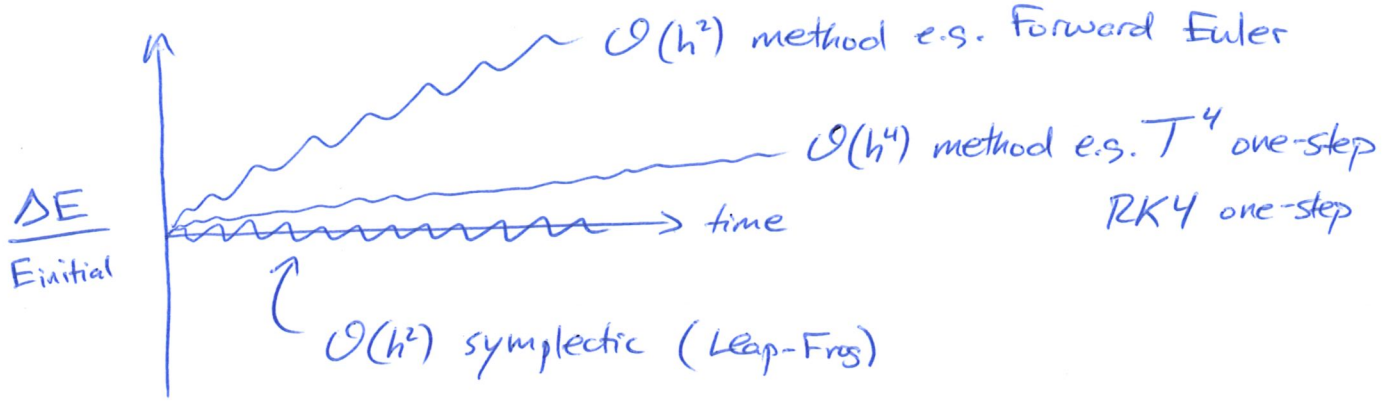
$$\Rightarrow \tilde{y}_{i+1} = \left(\frac{1}{1+\alpha h}\right) \tilde{y}_i$$

pre-factor is < 1 regardless of the value of the stepsize h

(and is positive given $\alpha > 0$)

$\Rightarrow$ unconditionally stable

build intuition for errors with different methods:



Runge-Kutta Methods

goal: high order $\mathcal{E}_{local}$ without needing $f^{(k)}(t,y)$

idea: replace with numerical quadrature (integration) from $t_i \to t_{i+1}$

$$\int_a^b f(x)\,dx \approx \sum_{i=0}^{N} a_i f(x_i)$$

$$= \frac{1}{2}h\left[f(a)+f(b)\right] - \frac{h^3}{12}f''(\xi) \qquad \text{Trapezoidal rule}$$

$$= \frac{1}{3}h\left[f(a)+4f\left(\frac{a+b}{2}\right)+f(b)\right] - \frac{h^5}{90}f^{(4)}(\xi) \qquad \text{Simpson's rule}$$

recall our ODE problem $y'(t) = f(t,y)$, $y(0)=y_0$

$$\Rightarrow y_{i+1} = y_i + \int_{t_i}^{t_{i+1}} f(t,y)\,dt$$

$$\Rightarrow \boxed{\tilde{y}_{i+1} = \tilde{y}_i + h\sum_{j=1}^{N} a_j f(t_{ij}, \tilde{y}_{ij})}$$

i.e. (1) evaluate $f(t,y)$ N times, at $t_{ij} \in [t_i, t_{i+1}]$

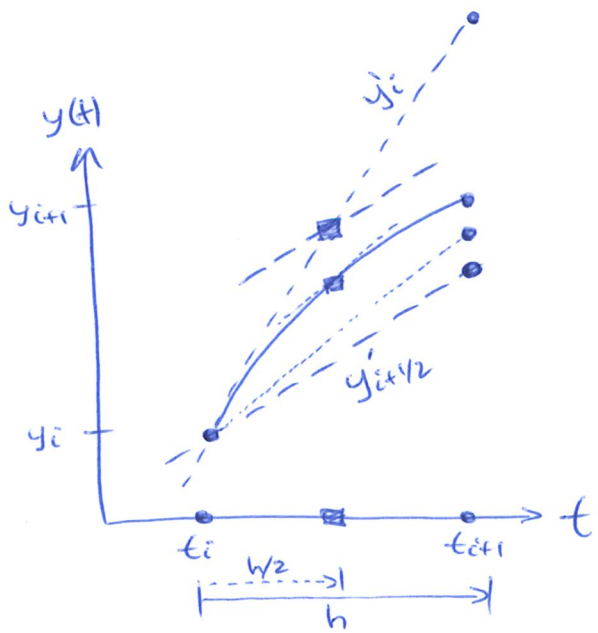
(2) sum with weights a_j

Ex: midpoint method

$$y_{i+1} = y_i + hf(t_{i+1/2}, \underbrace{y_{i+1/2}}_{\text{unknown!}})$$

$$= y_i + hf\left(t_i + \frac{h}{2}, y_i + \frac{h}{2}f(t_i,y_i)\right)$$

note: $N=1$, $a_j=1$, $t_{ij} = t_{i+1/2}$



Derivation for $\mathcal{O}(h^2)$

idea: obtain $\{a_j, t_{ij}\}$ via comparison with Taylor expansion

$$y(t_{i+1}) = y(t_i) + hy'(t_i) + \frac{h^2}{2}y''(t_i) + \mathcal{O}(h^3)$$

$$= y(t_i) + hf + \frac{h^2}{2}(f_t + ff_y) + \mathcal{O}(h^3)$$

notation: $f_t = \frac{\partial f}{\partial t}$

where $\{f, f_t, f_y\}$ are evaluated at (t_i, y_i)

- want weights & sample points such that Π approximates T^2 with $\tau \sim \mathcal{O}(h^2)$ or less

$\Rightarrow$ assume ansatz for the form of Π:

$$y_{i+1} = y_i + ha_1 f + ha_2 f(t_i + c_2 h, y_i + hb_{12} f(t_i, y_i))$$

$a_i \equiv$ quadrature weights

$b_{12} \equiv$ shift of value at evaluation

$c_2 \equiv$ shift of time for evaluation

recall multi-variate Taylor expansion:

$$f(x+h, y+k) = f(x,y) + hf_x(x,y) + kf_y(x,y) + \mathcal{O}(h^2)$$

expand the last term above:

$$f(t_i + c_2 h, y_i + hb_{12} f) = f + c_2 h f_t + hb_{12} f f_y$$

$$\Rightarrow y_{i+1} = y_i + ha_1 f + ha_2 [f + c_2 h f_t + hb_{12} f f_y] \quad \text{quadratic in } h$$

$$= y_i + hf(a_1 + a_2) + ha_2 [c_2 h f_t + hb_{12} f f_y]$$

has same form as T^2 if we let $c_2 = b_{12}$

$$\Rightarrow y_{i+1} = y_i + hf(a_1 + a_2) + h^2 a_2 b_{12} (f_t + f f_y)$$

and comparing coefficients:

$$\left\{ \begin{array}{l} a_1 + a_2 = 1 \\ a_2 b_{12} = 1/2 \end{array} \right\}$$ two equations, three unknowns $\Rightarrow$ one degree of freedom

$$\Rightarrow a_2 = \gamma$$

$$\Rightarrow \boxed{y_{i+1} = y_i + h\left[(1-\gamma) f(t_i, y_i) + \gamma f\left(t_i + \frac{h}{2\gamma}, y_i + \frac{h}{2\gamma} f(t_i, y_i)\right)\right]}$$

RK2: Runge-Kutta method of second order